Corey Zhou

coreyzh@gmail.com | (510) 329-7168 | linkedin.com/in/corey-z-2a7781231 | San Leandro, CA

OBJECTIVE

Computer engineering graduate seeking a Design Verification Engineer role. Hands-on experience verifying a RISC-V processor using constrained-random testing, SystemVerilog assertions, with self-directed study in UVM. Fast learner with strong programming and debugging fundamentals and a track record of taking ownership of complex technical problems.

TECHNICAL SKILLS

Hardware Description & Verification: SystemVerilog, UVM (coursework/self-study), Verilog, Constrained-Random Verification, SystemVerilog Assertions (SVA), Functional Coverage, MIPS32 Assembly

EDA / Simulation Tools: ModelSim, HSPICE, SUE

Programming Languages: C/C++, Python, Java, JavaScript/TypeScript

Scripting & Test Automation: Pytest, Selenium, Jest

Systems: Linux/Unix, Windows

Data & Machine Learning: TensorFlow, Keras, Scikit-learn, NumPy, Pandas, Matplotlib

Additional: Docker, Jenkins, Git, CI/CD, AWS, Terraform, Networking (TCP/IP, ARP)

EDUCATION

University of California, Santa Barbara — *B.S. Computer Engineering*
GPA: 3.78 | Dean's Honors

Relevant Self-Study (Udemy): SystemVerilog Verification Series (SystemVerilog testbenches & UVM fundamentals); Linux Administration Bootcamp; CI/CD with Jenkins & Docker; Terraform for AWS; The Ultimate MySQL Bootcamp; JavaScript: Understanding the Weird Parts

FUNCTIONAL VERIFICATION PROJECTS — UCSB

RISC-V Processor Functional Verification

- Built a SystemVerilog-based verification environment to detect functional bugs in a RISC-V processor pipeline
- Developed constrained-random assembly test programs to drive the DUT and exercise pipeline corner cases
- Wrote SystemVerilog assertions (SVA) to automatically check pipeline correctness during simulation
- Debugged simulation failures and traced root causes back through the pipeline to verify correct behavior

SAT-Based Monomial Learning for Automated Root-Cause Analysis

- Formulated root-causing a failure as a Boolean satisfiability (SAT) problem, learning a minimal monomial (AND-of-literals) expression consistent with one failing sample among many passing samples
- Encoded feature-sign and sample-consistency constraints in Python using the PyEDA SAT library, with a cardinality constraint to bound the number of literals and control classifier complexity
- Solved the resulting CNF formula to recover a compact logical expression identifying the feature combination responsible for the failure, directly transferable to root-causing functional coverage misses or bug signatures

EXPERIENCE

SlamOptix — Founding Engineer

- Joined as founding engineer with full ownership of architecture and technical direction, building core products end-to-end as the sole engineer (MongoDB, Node.js, Next.js, NGINX)
- Built an internal AI-assisted development pipeline (self-hosted LLM inference + automated test/integration) that reduced manual effort on routine engineering tasks
- Made independent technical trade-off decisions balancing scalability, cost, and development speed

Independent Contractor — Test Automation & Security

- Built automated test suites (Python, Pytest, Selenium) to validate web application functionality end-to-end; designed a full-stack device-monitoring app (Node.js, Next.js, MariaDB, Docker Compose)
- Performed protocol-level security testing (Kali Linux, Scapy, NMap, Wireshark) to identify DHCP and network-layer vulnerabilities, applying a systematic, spec-driven testing methodology directly transferable to hardware verification

CK-12 Foundation — Math Interactives Intern: Palo Alto, CA

- Designed interactive middle/high school math content and performed QA review on other interns' work using GeoGebra's JavaScript API

ADDITIONAL PROJECTS

- **Anchorless Autonomous Boat Capstone — UCSB:** 5-person, year-long capstone building an autonomous boat that holds station within a target radius via Raspberry Pi control software and cellular communication
- **Applied ML & Full-Stack Projects:** LSTM stock-price predictor (TensorFlow/Keras, NumPy/Pandas); subway route-planning app (MySQL, Flask, NGINX, Docker); self-taught Unreal Engine (C++/Blueprint) game prototypes